

sphere_ref_lib.py 取扱説明書

このドキュメントは [sphere_11/sphere_ref_lib.py](#) に含まれる関数群の概要・引数・返回值・注意点をまとめたものです。

- 想定用途: 球面（半径 R ）での鏡面反射の幾何光学（レイ）計算、観測球面（半径 R_2 ）上での角度分布ヒストグラム作成、反射率・補正の適用、netCDF（xarray）入出力。
- 角度の単位:
 - `ref_rays_count` 系は 度 (deg) を返す (`theta_deg, phi_deg`)。
 - `make_nc_inc` 内部の `inc_theta_arr, inc_phi_arr` は ラジアン のまま格納されている（要注意）。
- 座標系の前提: 反射体は原点中心の球。入射方向 d は 3次元ベクトル。 $p0=[x, y, z0]$ から d 方向に進む直線と球の交点を求める。

1. 出力先ディレクトリ

```
set_output_dir(out="./output/")
```

画像などの出力先ディレクトリを作成してパスを返します。

- 引数
 - `out` (str): 出力先ディレクトリ（デフォルト `./output/`）
- 返回值
 - `output_dir` (str): 作成済みのディレクトリパス

```
set_output_dir_nc(out_nc="./nc_underground/")
```

netCDF 出力用ディレクトリを作成してパスを返します。

- 引数
 - `out_nc` (str): 出力先ディレクトリ（デフォルト `./nc_underground/`）
- 返回值
 - `fp_nc` (str): 作成済みのディレクトリパス

2. 基本パラメータ

```
set_basic_params()
```

計算に用いる基本パラメータ（球半径、サンプリング刻み、入射方向など）を返します。

- 返回值
 - `R` (float): 球半径（デフォルト 1.0）
 - `step` (float): サンプリング刻み（デフォルト 0.0001）
 - `xs` (np.ndarray): x サンプル配列（0 から R まで）
 - `d` (np.ndarray shape=(3,)): 入射方向ベクトル（デフォルト `[0, 0, -1]`）
 - `z0` (float): 入射開始位置の z （デフォルト 1000.0）

- `R2` (float): 観測球面の半径（デフォルト 20000）

3. 光線追跡（反射の幾何）

```
ref_rays_count(R2, xs, d, Z0, R=1.0, y=0.0, print_info=False,
inc_on=False)
```

鏡面反射を仮定して、観測球面（半径 `R2`）に到達する反射光の角度（ θ, φ ）分布をサンプルします。

- 概要
 1. 入射直線 $p(t) = p_0 + t \cdot d$ と反射球 $|p| = R$ の交点を解く。
 2. 法線 $n = p / |p|$ から反射方向 $r = d - 2(d \cdot n)n$ を計算。
 3. 反射直線 $p_2(t) = p + t \cdot r$ と観測球 $|p_2| = R_2$ の交点を解く。
 4. 観測点 p_2 を単位化して、天頂角 $\theta = \arccos(z)$ と方位角 $\phi = \arctan_2(y, x)$ を算出。
- 引数
 - `R2` (float): 観測球面半径
 - `xs` (np.ndarray): 入射点の x サンプルング ($p_0 = [x, y, Z_0]$)
 - `d` (np.ndarray): 入射方向ベクトル
 - `Z0` (float): 入射開始点の z
 - `R` (float): 反射球半径（デフォルト 1.0）
 - `y` (float): 入射開始点の y （デフォルト 0.0）
 - `print_info` (bool): サンプル数などを表示
 - `inc_on` (bool): 入射側（裏側交点）も併せて角度リストへ入れるモード
- 返り値
 - `theta_arr_r2` (np.ndarray): 角度 θ (deg)
 - `phi_arr_r2` (np.ndarray): 角度 ϕ (deg, 範囲 $[0, 360)$)
 - `p0s` (list[np.ndarray]): 入射開始点 p_0
 - `p2s` (list[np.ndarray]): 観測球面交点 p_2
 - `p_hits` (list[np.ndarray]): 反射点 p (`inc_on` の場合、条件によって裏側交点も追加される)
 - `ref_dirs` (list[np.ndarray]): 反射方向 r
- 注意点
 - `inc_on=True` の場合、`p_hits` と角度配列に「裏側交点」が混ざることがあります。後段処理が「反射点のみ」を想定していると破綻します。
 - `inc_on` での「掩蔽判定」は `np.linalg.norm([pinc[0], pinc[1], 0]) <= R` を用いています（意味としては“円柱投影で球に当たるか”の判定）。用途に応じて妥当性確認が必要です。

```
abs_check_r2(R2, p2s)
```

観測球面交点 `p2s` が半径 `R2` の球面上にあるか（距離誤差が閾値以下か）を検証します。

- 引数
 - `R2` (float): 期待半径
 - `p2s` (array-like shape=(N,3)): 観測点座標列

- 返り値
 - `result_check` (bool): 全点が閾値内なら True

4. 図示・角度分布ヒストグラム

```
plot_rays_hist_2d(R2s, xs, d, Z0, p0s, p2s, p_hits, ref_dirs, fp,
R=1.0)
```

1. x-z 平面 ($y=0$) で入射線と反射線を描画し、2) `R2s` ごとの `theta` ヒストグラム (1D) を描画します。

- 引数
 - `R2s` (list[float]): 観測半径のリスト
 - `xs, d, Z0, R`: `ref_rays_count` と同様
 - `p0s, p2s, p_hits, ref_dirs`: 事前計算済みの配列（ただし本関数内で再度 `ref_rays_count` を呼ぶため、最初の描画以外では必須ではない）
 - `fp` (str): 保存先ディレクトリ（末尾 / 推奨）
- 出力
 - `inc_ref_rays_Rmix.png`
 - `theta_hist_Rmix.png`
- 注意点
 - 反射線描画の色付け `ic` の算出が `p_hits` 長さに依存しています。`p_hits` が `xs` と 1対1対応していない場合（`inc_on` 等）に破綻します。

5. netCDF（反射角分布）作成

```
make_nc_sphere(R2, step, xs, d, Z0, fp_nc)
```

`y` を走査しながら反射角分布 (`theta, phi`) を計算し、`reflected_rays_R2_{R2}.nc` として保存します。

- 引数
 - `R2` (float): 観測球半径
 - `step` (float): `y` 走査の基準刻み (`y_array = arange(-1, 1, step*20)`)
 - `xs, d, Z0`: `ref_rays_count` と同様
 - `fp_nc` (str): 出力ディレクトリ
- 出力
 - `reflected_rays_R2_{...}.nc` (xarray Dataset)
- 生成データの構造
 - `theta(x) / phi(x)` を持つ Dataset を `y` 次元で concat

- coords:
 - `x`: `p_hits` の `x` 座標 (※「反射点 index」ではない)
 - `y`: スカラー `y`
- 注意点
 - `x` 座標が重複しうるため、`xr.concat(join="outer")` の振る舞い (欠損埋め) が意図通りか確認してください。

```
make_nc_inc(R2, step, xs, d, Z0, fp_nc)
```

入射波の角度分布を作り、`incident_rays_norm_{R2}.nc` として保存します (実装コメントにもある通り、考え方が異なる可能性あり)。

- 引数
 - `R2` (float): “基準球” 半径として使われる
 - `step, xs, d, Z0, fp_nc`: 同様
- 出力
 - `incident_rays_norm_{R2}.nc`
- 注意点 (重要)
 - Dataset の `theta/phi` は **ラジアン** で格納されます (`np.degrees` していない)。他の関数群は度が多いので混同注意。
 - `ref_rays_count(norm_R2*2, ..., R=norm_R2, inc_on=True)` の戻り `p_hits` が「半径 `norm_R2` の球面上」とみなせる前提で `abs_check_r2(norm_R2, p_hits)` を行っています。`inc_on` で `p_hits` に混入する要素の意味に注意してください。

6. netCDF 読み込み

```
load_nc_sphere(R2, fp_nc)
```

`reflected_rays_R2_{R2}.nc` を読み込みます。

- 引数
 - `R2` (float | str): ファイル名に埋め込む値
 - `fp_nc` (str): ディレクトリ
- 返回值
 - `ds_all` (xr.Dataset)

```
load_nc_inc(R2, fp_nc)
```

`incident_rays_norm_{R2}.nc` を読み込みます。

- 引数/返回值
 - 上に同じ (Dataset)

7. 入射強度の見積

```
calc_inc_field(R_norm, y_step=20)
```

`xs` と `ys` のサンプリング密度から、単位立体角（1deg×1deg 相当）あたりの入射レイ密度を概算します。

- 引数
 - `R_norm` (float): スケーリング半径（観測半径等）
 - `y_step` (int): `ys = arange(-1, 1, step*y_step)` の係数
 - 返回值
 - `sum_ray` (int): 総レイ数 (`len(xs)*len(ys)`)
 - `sum_field` (float): サンプリング面積 (`(xs[-1]-xs[0])*(ys[-1]-ys[0])`)
 - `unit_inc` (float): 1deg×1deg を仮定した入射密度（概算）
-

8. 角度分布 → 2Dヒートマップ

```
make_thph_2dhist(ds_all)
```

Dataset (`theta`, `phi`) をフラット化し、(`theta`, `phi`) の2Dヒストグラムを作成して `xr.DataArray` で返します。

- 引数
 - `ds_all` (xr.Dataset): `theta` と `phi` を含む
- 返回值
 - `heat_da` (xr.DataArray): `dims= ("theta", "phi"), name=counts`
- 注意点
 - ここで使うピンは `theta: 0..180 (1deg)`, `phi: 0..360 (1deg)` 固定。
 - 入力 of 角度単位が 度である前提 (`np.linspace(0, 180, 181)` に入れる)。 `make_nc_inc` の出力 (ラジアン) を直接入れると壊れます。

```
draw_hist2d(heat_da, R2)
```

`heat_da` を `imshow` で表示します（保存はしない）。

- 引数
 - `heat_da` (xr.DataArray)
 - `R2` (float): タイトル表示用
-

9. 反射率（Fresnel）関連

```
get_reflection_rate(d, n, e0, e1)
```

入射ベクトル `d` と法線 `n` から入射角を計算し、TE/TM および平均反射率を返します。

- 引数

- `d` (np.ndarray): 入射方向ベクトル
- `n` (np.ndarray): 面法線（単位ベクトル想定）
- `e0` (float): 入射側比誘電率（通常 1）
- `e1` (float): 透過側比誘電率
- 返回值
 - `Rtm` (float): TMモード反射率
 - `Rte` (float): TEモード反射率
 - `Rave` (float): $(|Rtm| + |Rte|) / 2$
 - `theta_s` (float): 入射角（rad）

`get_reflection_rate_angle(theta_s, e0, e1)`

入射角を直接与えて反射率を返します。

- 引数
 - `theta_s` (float): 入射角（rad）
 - `e0, e1` (float): 比誘電率
- 返回值
 - `Rtm, Rte, Rave` (float)

`get_sc_angle(theta_s, R_moon, H_obs)`

入射角から探査機が捕捉する角度（アルファ）を計算します。

- 引数
 - `theta_s` (float): 入射角（rad）
 - `R_moon` (float): 天体半径
 - `H_obs` (float): 観測高度
- 返回值
 - `theta_al` (float): 角度（rad）

10. 反射率付きレイトレーシング

`ref_rays_count_ref(R2, xs, d, Z0, R, y=0.0, print_info=True, reflectance=False, e1=3.0)`

`ref_rays_count` に「反射率計算」と「探査機角度」を付加した拡張版です。

- 引数
 - `reflectance`:
 - 1 のとき反射率計算を実施し、`amp_arr / alp_arr` を返す
 - それ以外では空配列
 - `e1` (float): 透過側比誘電率（反射率計算に使用）
 - その他は `ref_rays_count` と同様
- 返回值
 - `theta_arr_r2, phi_arr_r2, p0s, p2s, p_hits, ref_dirs`（同様）

- `amp_arr` (np.ndarray shape=(N,3)): [Rtm,Rte,Rave] (`reflectance==1` の場合)
- `alp_arr` (np.ndarray shape=(N,)): 探査機角度 (rad想定、`get_sc_angle` に依存)
- 注意点
 - `reflectance==1` 以外では `amps=[]` のままなので、`amp_arr` は空 (shape=(0,)) になります。

11. 天体パラメータ

`get_default_param(target)`

`target` に応じて (月 / ガニメデ) パラメータを返します。

- 引数
 - `target` (str): "moon" または "ganymede"
- 返回值
 - `e1, e2, H_obs, D_moon, R_moon, e1, e2, tandelta`
- 注意点 (重要)
 - `e1, e2` が **重複して2回**返ります (呼び出し側もそれに合わせた代入をしています)。APIとしてはやや紛らわしいため、利用時は代入順を崩さないでください。
 - ↑2026/3/30に修正。二回目の`e1, e2`を消去しました。これにより以前に書いたこの関数の呼び出しでエラーが出ます。適宜修正を。

12. φ 方向の積分 (θ 1D化)

`sum_phi(heat_da, phi_min, phi_max)`

`phi` 範囲を切り出して `phi` 方向に和を取り、`theta` だけの 1D DataArray を返します。

- 引数
 - `heat_da` (xr.DataArray): dims に `phi` と `theta` を含む
 - `phi_min, phi_max` (float): φ の範囲 (deg)
- 返回值
 - `theta_1d_counts_phi_0_90` (xr.DataArray): `phi` を sum 済み

13. 補正 (Steve correction)

このライブラリには補正の適用方法が2系統あります。

- 旧: `steve_correction(..., *args)` (フラグ配列で分岐)
- 新: `steve_correction_pipeline(heat_da, params, corrections)` (パイプライン方式)

```
steve_correction(target, R2, fp_nc, theta_arr_r2, phi_arr_r2, p0s,
p2s, p_hits, ref_dirs, amp_arr, alp_arr, heat_da, *args)
```

3種類の補正（立体角・反射率・垂直方向）をフラグでON/OFFして適用します。

- 引数
 - `args[0]`: 1なら I（立体角補正）
 - `args[1]`: 1なら II（反射率補正）
 - `args[2]`: 1なら III（垂直方向補正）
 - その他: 各補正に必要なデータ一式
- 返り値
 - `fin_da` (xr.DataArray): 補正後
- 注意点
 - `args` が不足していると `IndexError` になります（防御無し）。

```
i_the_solid_angle(heat_da)
```

立体角補正として `sin(theta)` で割ります。

- 返り値
 - `ste_da` (xr.DataArray): `counts per sinθ`

```
i2_the_solid_angle_field(R2, heat_da)
```

立体角補正に加え、焦点距離 `Rf=R2-0.5` を用いて単位面積あたりへ正規化します（`Rf^2` で割る）。

```
i3_the_solid_angle_field_upd(R2, heat_da)
```

`i2` の改良版。`saf_clear_standard` によって θ ごとに面積比補正係数を計算して適用します。

```
saf_clear_standard(R2, heat_da, f=0.5)
```

`i3` 用の内部計算。 θ ごとに `Rf, thf, thr` を返します。

- 返り値
 - `Rfs` (np.ndarray): 焦点からの距離
 - `thfs` (np.ndarray): 有効角度幅（モデル由来）
 - `thrs` (np.ndarray): 面積比補正

```
ii_the_reflection_rate(target, R2, ste_da, fp_nc, apply_ref="average",
fn="alpha_to_theta_forhist_ver2.nc")
```

探査機角度 θ (deg) → 入射角 (deg) 変換テーブル (netCDF) を使い、Fresnel反射率を `ste_da` に乗じます。

- 引数
 - `apply_ref`: "average" | "TE" | "TM"

- `fn`: 角度変換テーブル (DataArray) のファイル名
- 注意点
 - `R2_for = R2 * 1000` として変換テーブルの `H` と照合します (慣例とのこと)。`R2` の単位系が混在しやすいので注意。
 - `R2==1000000` は“無限遠”扱いで入射角を `theta/2` と近似します。

```
iii_the_vertical_direction(p2s, ref_dirs, theta_arr_r2, ste_da_ref)
```

観測点方向ベクトルと反射方向ベクトルの内積 (cos) を θ ビンごとに平均化し、`ste_da_ref` を cos で割って補正します。

- 注意点
 - `ve_da.fillna(0)` は 代入されていないため、NaN は残ります (xarray の `fillna` は新しいオブジェクトを返す)。

14. 探査機角度 $\leftrightarrow$ 入射角 変換テーブル作成

```
calc_alpha(ts, H, R)
```

入射角 `ts` (deg) から探査機角 `alpha` (rad $\rightarrow$ deg変換込み) を計算して deg で返します。`H`は地表からの高度であることに注意。

```
calc_alpha_opt(ts, H, R, alpha_target)
```

`calc_alpha(ts,...) - alpha_target` を返す目的関数 (数値解法用)。

```
make_nc_alpha_ts(hs, fp_nc, fn="alpha_to_theta_forhist.nc")
```

高度配列 `hs` と `alpha` ターゲット (0.5..179.5 deg) に対して、対応する入射角 `theta_s` を `fsolve` で求めて netCDF 保存します。

- 出力
 - `fp_nc/fn`

```
load_nc_alpha_ts(fp_nc, fn="alpha_to_theta_forhist.nc")
```

上のテーブルを `xr.open_dataarray` で読み込みます。

15. 補正パイプライン (推奨)

```
steve_correction_pipeline(heat_da, params, corrections,
print_info=True)
```

補正を (補正名, 補正ごとの引数dict) のリストとして順番に適用します。

- 引数
 - `heat_da` (xr.DataArray): 入力

- `params` (dict): 共有パラメータ（補正関数内部で参照）
- `corrections` (list[tuple]): 例 `[("solid_angle", {}), ("reflection_rate", {"apply_ref": "average"})]`
- `print_info` (bool): 適用ログを表示

- 返り値

- `result_da` (xr.DataArray)

- 利用可能な補正名

- `solid_angle`
- `reflection_rate`
- `vertical_direction`
- `solid_angle_field`
- `solid_angle_field_upd`

```
i_the_solid_angle_adapted(data_da, params)
```

```
i2_the_solid_angle_field_adapted(data_da, params)
```

```
i3_the_solid_angle_field_upd_adapted(data_da, params)
```

```
ii_the_reflection_rate_adapted(data_da, params, apply_ref="average",
fn="alpha_to_theta_forhist_ver2.nc")
```

```
iii_the_vertical_direction_adapted(data_da, params)
```

上記の各補正関数を「統一インターフェース」に合わせたラッパーです。

16. 典型的な使用例

例1: 反射角分布を計算して2Dヒスト作成

```
from sphere_ref_lib import set_basic_params, ref_rays_count,
make_thph_2dhist

R, step, xs, d, Z0, R2 = set_basic_params()

theta_deg, phi_deg, p0s, p2s, p_hits, ref_dirs = ref_rays_count(R2, xs, d,
Z0, R=R, y=0.0)

import xarray as xr
# theta/phi を Dataset 化 (簡易)
ds = xr.Dataset({
    "theta": ("x", theta_deg),
    "phi": ("x", phi_deg),
})
heat_da = make_thph_2dhist(ds)
```

例2: 補正パイプライン適用

```
params = {
    "target": "moon",
    "R2": R2,
    "fp_nc": "./nc_underground/",
    "theta_arr_r2": theta_deg,
    "phi_arr_r2": phi_deg,
    "p0s": p0s,
    "p2s": p2s,
    "p_hits": p_hits,
    "ref_dirs": ref_dirs,
    "amp_arr": None,
    "alp_arr": None,
}

corrections = [
    ("solid_angle", {}),
    ("reflection_rate", {"apply_ref": "average"}),
    ("vertical_direction", {}),
]

from sphere_ref_lib import steve_correction_pipeline
result = steve_correction_pipeline(heat_da, params, corrections)
```

付録: 既知の注意・改善候補（挙動確認推奨）

- `make_nc_inc` は角度単位（rad/deg）が他と異なる可能性が高い。
- `get_default_param` は戻り値が重複しており、呼び出し側は現状の順に依存。
- `iii_the_vertical_direction` の `fillna(0)` が代入されていないため NaN が残る可能性がある。